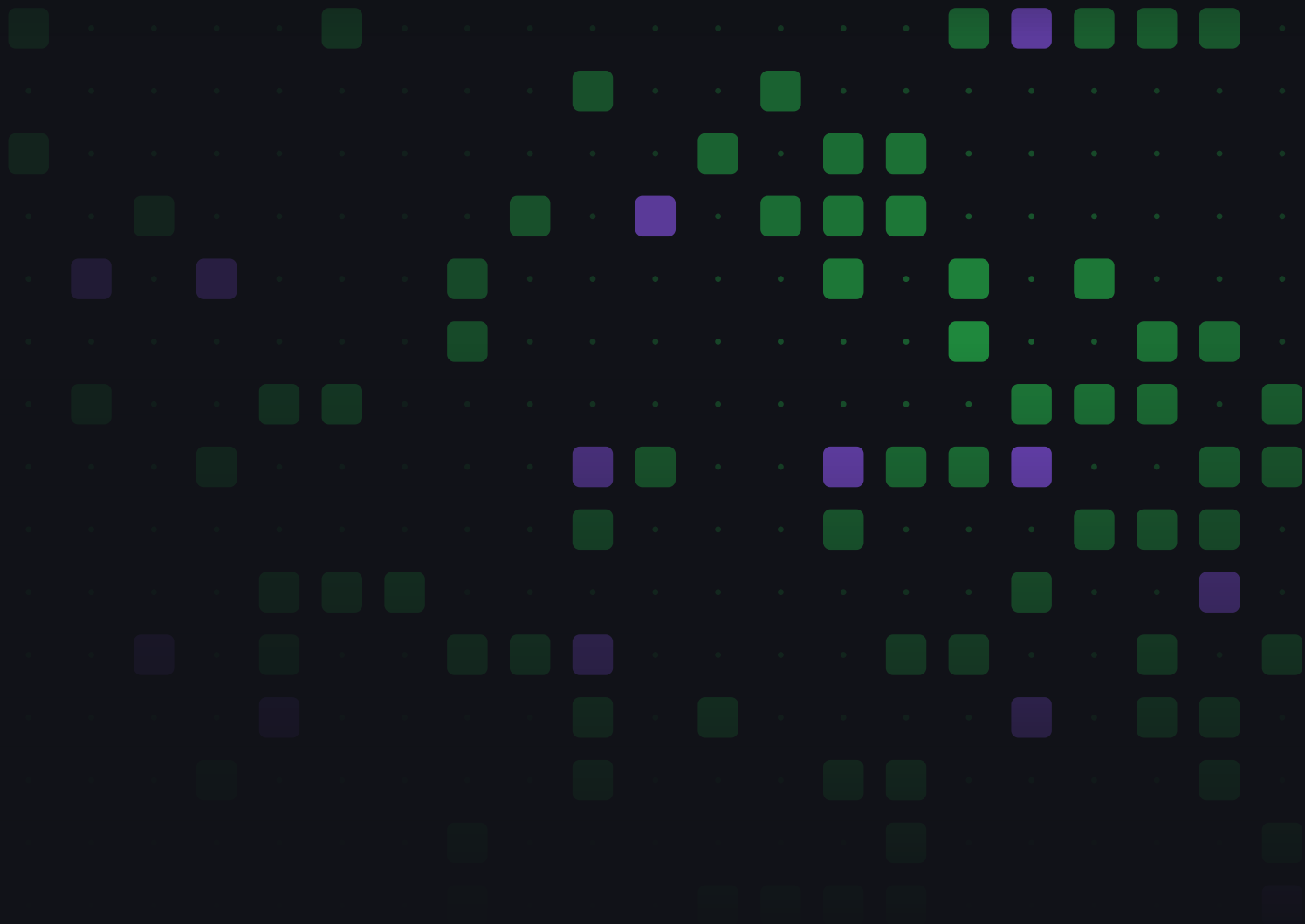


A BEGINNER'S MANUAL

GitHub

Save, share, and collaborate on your work, with Git explained from zero

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

GitHub

Save, share, and collaborate on your work, with Git explained from zero

GitHub is where most of the world's software lives, and it's where AI coding tools like Claude Code, Cursor, and Copilot expect your projects to be. It keeps a complete history of your work, lets you undo almost any mistake, and makes it easy to work with other people (and AI agents) without stepping on each other's toes.

This manual starts with the ideas, moves on to clicking around the GitHub website with nothing installed, and ends with the everyday commands, publishing a website, automation, and keeping your account secure.

HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

Contents

01 Welcome to GitHub 5

1.1 Git and GitHub are two different things 5

1.2 Why bother? 5

1.3 Words you'll hear constantly 6

02 Create Your Account 7

2.1 Sign up 7

2.2 Turn on two-factor authentication 7

2.3 Make your profile friendly 8

03 Your First Repository, Entirely in the Browser 9

3.1 Create a repository 9

3.2 Edit a file and make your first commit 9

3.3 Upload files 10

3.4 Writing in Markdown 10

04 Install Git on Your Computer 11

4.1 Install Git 11

4.2 Tell Git who you are 11

4.3 Option A: GitHub Desktop (easiest) 12

4.4 Option B: the GitHub CLI (best for AI tools) 12

05 The Everyday Workflow 13

5.1 The three places your work lives 13

5.2 Get a repository onto your computer 13

5.3 Save and share changes 14

5.4 Writing good commit messages 15

5.5 Looking back in time 15

5.6 Ignoring files with .gitignore 15

06 Branches and Pull Requests 16

6.1 The branch-and-PR workflow 16

6.2 Three ways to merge 17

6.3 Merge conflicts, without panic 17

6.4 Protect your main branch 18
6.5 Contributing to someone else's project 18

07 **Issues, Projects, and Discussion** 19
7.1 Issues 19
7.2 Projects 19
7.3 Notifications and mentions 19

08 **Publish a Website with GitHub Pages** 20

09 **Automate with GitHub Actions** 21

10 **GitHub and AI** 23
10.1 GitHub Copilot 23
10.2 Claude Code and other agents 23
10.3 Codespaces: a computer in the cloud 24

11 **Security and Good Habits** 25
11.1 Never commit secrets 25
11.2 Personal access tokens 25
11.3 Keep dependencies healthy 26
11.4 Checklist for a healthy repository 26

12 **Plans and Pricing** 27

13 **Troubleshooting** 28

Quick Reference 29

Glossary 30

CHAPTER 1

Welcome to GitHub

You've probably heard "push it to GitHub" or "it's in the repo." This chapter explains what those words mean and why GitHub is worth learning, even if you never plan to call yourself a programmer.

IN THIS CHAPTER

- The difference between Git and GitHub
- Why GitHub matters, especially when you work with AI
- The handful of words you'll hear constantly

1.1 Git and GitHub are two different things

Git is a free program that runs on your computer. It watches a folder and records snapshots of the files in it over time, so you can see what changed, when, and why, and go back to any earlier version. Git was created in 2005 and is used almost everywhere.

GitHub is a website (github.com, owned by Microsoft) that stores copies of Git projects online. It adds everything you need to work with others: a web interface, discussions, task tracking, reviews, automation, and website hosting.

> NOTE
>
> A good analogy: Git is like the "track changes" and version history in a word processor; GitHub is like a shared cloud drive built around that history.

1.2 Why bother?

- **A safety net.** Every saved snapshot is permanent. Broke something? Go back.
- **A backup.** Your work lives on GitHub's servers as well as your computer.

- **A shared workspace.** Several people, or several AI agents, can work on the same project at once.
- **A portfolio.** Your public projects show what you can build.
- **AI tools expect it.** Claude Code, Copilot, Cursor, Lovable, Bolt, and others connect to GitHub to read, change, and publish your code.

1.3 Words you'll hear constantly

WORD	PLAIN-ENGLISH MEANING
Repository (repo)	A project folder that Git tracks, plus its entire history
Commit	A saved snapshot of your files, with a message describing the change
Branch	A separate line of work, so you can try things without touching the main version
Main	The default branch: the "official" version of the project
Merge	Combine the changes from one branch into another
Pull request (PR)	A proposal to merge a branch, with a place to discuss and review it
Issue	A to-do item, bug report, or idea, tracked on GitHub
Clone	Download a full copy of a repository to your computer
Push / Pull	Send your commits up to GitHub / bring others' commits down
Fork	Your own copy of someone else's repository on GitHub

Don't worry about memorizing these; each one gets its own explanation as we go, and they're all in the Glossary.

CHAPTER 2

Create Your Account

2.1 Sign up

CREATE A FREE GITHUB ACCOUNT

1. **Go to github.com.**
   Click **Sign up**.
2. **Enter your email, a password, and a username.**
   Your username becomes part of your web address (github.com/*username*), so pick something you're happy to share professionally.
3. **Verify.**
   Solve the quick puzzle and enter the code GitHub emails you.
4. **Choose the Free plan.**
   It includes unlimited public and private repositories. You can upgrade later.

> **TIP**
> Choose a username that's short and easy to say aloud. Changing it later breaks old links to your work.

2.2 Turn on two-factor authentication

Two-factor authentication (2FA) means signing in needs your password *plus* a second proof, such as a code from your phone. GitHub requires 2FA for anyone who contributes code, so set it up now.

ENABLE 2FA

1. **Open settings.**
   Click your profile picture (top right), then **Settings**.

2. **Find the security page.**
   Go to `Password and authentication`.

3. **Enable two-factor authentication.**
   Click **Enable two-factor authentication**.

4. **Scan the QR code** with an authenticator app such as Google Authenticator, Microsoft Authenticator, 1Password, or Authy, and type the 6-digit code it shows.

5. **Save your recovery codes.**
   Download them and keep them somewhere safe, offline. They're your only way back in if you lose your phone.

6. **Add a passkey (optional, recommended).**
   A **passkey** lets you sign in with your fingerprint, face, or device PIN instead of typing a password.

> **DANGER**
> Lose your phone *and* your recovery codes, and GitHub support usually cannot restore your account. Print the recovery codes and store them with your important papers.

2.3 Make your profile friendly

Click your profile picture, then **Your profile**, then **Edit profile** to add a photo, a short bio, and links. Optional but nice: create a repository named exactly the same as your username, and its README appears at the top of your profile page.

CHAPTER 3

Your First Repository, Entirely in the Browser

You can do real work on GitHub without installing anything. Let's create a repository, edit a file, and save a commit, all from the website.

3.1 Create a repository

NEW REPOSITORY

1. **Click the + menu** (top right) and choose **New repository**.
2. **Name it.**
   For example `my-first-repo`. Use lowercase letters and dashes.
3. **Add a description**, like "Learning GitHub."
4. **Choose Public or Private.**
   Public means anyone can see it; private means only you and people you invite.
5. **Tick "Add a README file."** A README is the welcome page of your repository.
6. **Click Create repository.**

You're now looking at your repository's home page. The file list is in the middle, the README is shown below it, and tabs across the top lead to **Issues**, **Pull requests**, **Actions**, and **Settings**.

3.2 Edit a file and make your first commit

EDIT AND COMMIT

1. **Open README.md** by clicking its name.

2. **Click the pencil icon** (Edit this file).
3. **Change the text.**
   Add a line such as `This is where I'm learning GitHub!`
4. **Click Commit changes…** (top right).
5. **Write a commit message**, a short note saying what you did: `Add welcome line to README`.
6. **Choose "Commit directly to the main branch"** and click **Commit changes**.

Congratulations: that's a **commit**. Click the clock icon labeled **commits** on the repository page to see the history, including GitHub's original "Initial commit."

3.3 Upload files

On the repository page, click **Add file**, then **Upload files**, and drag in documents, images, or code. Write a commit message and click **Commit changes**.

> **TIP**
> Press the `.` (period) key on any repository page to open **github.dev**, a full code editor that runs in your browser. It's great for editing several files at once.

3.4 Writing in Markdown

README files, issues, and comments on GitHub use **Markdown**, a simple way to format text with symbols:

MARKDOWN

```
# Big heading
## Smaller heading

**bold**, *italic*, and `code`

- a bullet
- another bullet

1. a numbered step
2. another step

[a link](https://github.com)
- [ ] a checkbox to tick off
```

CHAPTER 4

Install Git on Your Computer

Working in the browser is great for small edits. For real projects, and for AI coding tools that work on your computer, you'll want Git installed locally. You can drive it three ways: a friendly app (**GitHub Desktop**), the **command line**, or let an **AI agent** run the commands for you.

4.1 Install Git

SYSTEM	HOW
Windows	Download the installer from git-scm.com and accept the defaults. It includes **Git Bash** and **Git Credential Manager**, which handles signing in to GitHub.
macOS	Open Terminal and type `git --version`. If Git isn't installed, macOS offers to install the developer tools. Or use Homebrew: `brew install git`.
Linux	Ubuntu/Debian: `sudo apt install git` · Fedora: `sudo dnf install git`

Check it worked:

TERMINAL

```
git --version
```

4.2 Tell Git who you are

Every commit is labeled with a name and email. Set them once:

TERMINAL

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

> **TIP**
> To keep your real email private, use the **noreply** address GitHub gives you. Find it at `Settings` › `Emails` (it looks like `12345+username@users.noreply.github.com`) and tick **Keep my email addresses private**.

4.3 Option A: GitHub Desktop (easiest)

Download **GitHub Desktop** from desktop.github.com (Windows and Mac). Sign in with your GitHub account, and it handles authentication for you. Every action in the next chapter has a button in Desktop: **Clone**, **Commit**, **Push origin**, **Fetch**, **Create branch**, **Create pull request**.

4.4 Option B: the GitHub CLI (best for AI tools)

The **GitHub CLI**, a program called `gh`, lets you, and AI agents like Claude Code, do GitHub things from the terminal: create repositories, open pull requests, manage issues.

SYSTEM	INSTALL COMMAND
macOS	`brew install gh`
Windows	`winget install --id GitHub.cli` (or the installer from cli.github.com)
Linux	Follow the apt/dnf instructions linked from cli.github.com

Then sign in. It also sets up Git to use your GitHub login:

TERMINAL

```
gh auth login
```

Answer the questions: choose **GitHub.com**, **HTTPS**, **Yes** to authenticate Git, and **Login with a web browser**. Copy the one-time code, paste it into the browser page that opens, and approve.

> **NOTE**
> GitHub stopped accepting your account password for Git operations years ago. You sign in through the browser (GitHub Desktop, `gh auth login`, or Git Credential Manager), or with an SSH key or personal access token. More on tokens in Chapter 10.

CHAPTER 5

The Everyday Workflow

Almost all Git work is the same loop: **change files → stage → commit → push**. Here it is with the command line; GitHub Desktop has a button for each step.

5.1 The three places your work lives

PLACE	WHAT IT IS
Working folder	The files you see and edit on your computer
Staging area	Changes you've marked as "ready to go in the next commit"
Repository	The saved history of commits, both on your computer (*local*) and on GitHub (the *remote*, usually named `origin`)

5.2 Get a repository onto your computer

CLONE A REPOSITORY

1. **Copy the address.**
   On the repository page, click the green **Code** button and copy the HTTPS URL.
2. **Open a terminal** in the folder where you keep projects.
3. **Clone it.**

TERMINAL

```
git clone https://github.com/your-username/my-first-repo.git
cd my-first-repo
```

Or, with the GitHub CLI: `gh repo clone your-username/my-first-repo`.

Starting from a folder you already have instead? Turn it into a repository and publish it in one go:

TERMINAL

```
cd my-project
git init
gh repo create my-project --private --source=. --push
```

5.3 Save and share changes

THE DAILY LOOP

1. **See what changed.**

   TERMINAL

   ```
   git status
   ```

2. **Stage the changes you want to save.**
   A dot means “everything in this folder.”

   TERMINAL

   ```
   git add .
   ```

3. **Commit with a clear message.**

   TERMINAL

   ```
   git commit -m "Add contact page"
   ```

4. **Push to GitHub.**

   TERMINAL

   ```
   git push
   ```

5. **Get other people’s changes** before you start work next time.

   TERMINAL

   ```
   git pull
   ```

5.4 Writing good commit messages

A commit message is a note to your future self. Make it short and specific, and start with a verb:

- Good: `Fix crash when cart is empty` · `Add opening hours to homepage`
- Not so good: `stuff` · `changes` · `asdf`

5.5 Looking back in time

COMMAND	WHAT IT SHOWS
`git log --oneline`	A compact list of past commits
`git diff`	Exactly what you changed since the last commit
`git show <commit-id>`	The details of one commit
`git restore <file>`	Throw away uncommitted changes to a file
`git revert <commit-id>`	Create a new commit that undoes an old one, safely

> ⚠ **WARNING**
>
> Be careful with commands that contain `--force`, `reset --hard`, or `clean`. They can permanently discard work. If an AI agent wants to run one, read what it's about to do first.

5.6 Ignoring files with .gitignore

Some files should never go into a repository: passwords, huge downloads, and temporary files. List them in a file named `.gitignore` at the top of your project:

.GITIGNORE

```
# secrets
.env
# dependencies (re-downloadable)
node_modules/
# system junk
.DS_Store
```

GitHub offers ready-made `.gitignore` templates when you create a repository.

CHAPTER 6

Branches and Pull Requests

Branches let you work on something new without risking the version that works. **Pull requests** are how you propose, review, and merge that work. This is how professional teams, and AI coding agents, work every day.

6.1 The branch-and-PR workflow

MAKE A CHANGE ON A BRANCH

1. **Create and switch to a new branch.**

   TERMINAL

   ```
   git switch -c add-menu-page
   ```

2. **Make your changes, then commit them** as usual (`git add .` and `git commit -m "…"`).

3. **Push the branch to GitHub.**

   TERMINAL

   ```
   git push -u origin add-menu-page
   ```

4. **Open a pull request.**
   GitHub shows a yellow banner with **Compare & pull request**. Click it. Or run `gh pr create`.

5. **Describe the change.**
   Write what you did and why. Click **Create pull request**.

6. **Review.**
   Teammates (or AI reviewers) comment on specific lines, and automated checks run.

7. **Merge.**
   When everyone's happy, click **Merge pull request**, then **Delete branch**.

8. **Update your computer.**

```
TERMINAL
git switch main
git pull
```

> **TIP**
> Even if you work alone, use pull requests for anything an AI agent writes. The PR page gives you a clean, line-by-line view of every change before it reaches `main`.

6.2 Three ways to merge

When you merge a pull request, GitHub offers:

OPTION	WHAT HAPPENS	USE WHEN
Create a merge commit	Keeps every commit plus a "merge" commit	You want the full history
Squash and merge	Combines all the PR's commits into one	You want a tidy history (popular)
Rebase and merge	Replays commits on top of main, no merge commit	Advanced teams with linear history

6.3 Merge conflicts, without panic

A **merge conflict** happens when two branches changed the same lines differently, and Git can't decide which to keep. GitHub (or your editor) shows the conflict like this:

```
TEXT
<<<<<<< main
Open 7am to 3pm
=======
Open 8am to 4pm
>>>>>>> update-hours
```

Delete the markers, keep the right text (or a mix), save, then commit. In a pull request you can often click **Resolve conflicts** and fix it in the browser. Or ask your AI agent: "resolve the merge conflicts on this branch and explain your choices."

6.4 Protect your main branch

In a repository's `Settings` › `Rules` › `Rulesets` (or **Branches** on older layouts) you can require that changes to `main` go through a pull request, that checks pass, or that someone approves first. This is a great safety net when AI agents push code.

6.5 Contributing to someone else's project

You can't push to other people's repositories directly. Instead:

1. Click **Fork** to make your own copy.
2. Clone your fork, make a branch, commit, and push.
3. Open a pull request from your fork back to the original project.

CHAPTER 7

Issues, Projects, and Discussion

GitHub isn't just for code. Its planning tools work for any project.

7.1 Issues

An **issue** is a tracked item: a bug, a task, a question, or an idea. Click the **Issues** tab, then **New issue**. Give it a title and description, and optionally:

- **Assignees**: who's responsible (this can be an AI agent such as Copilot).
- **Labels**: colored tags like `bug`, `design`, or `good first issue`.
- **Milestone**: group issues toward a goal or deadline.

Write `Fixes #12` in a pull request's description and issue #12 closes automatically when the PR merges.

7.2 Projects

Projects are flexible boards and spreadsheets built from your issues and pull requests, like a lightweight Trello or Notion database. From your profile or organization, open **Projects**, then **New project**, and choose a **Board** (columns like *To do*, *In progress*, *Done*), a **Table**, or a **Roadmap**.

7.3 Notifications and mentions

Type `@username` in any comment to notify that person. Click the bell icon to see your notifications. **Watch** a repository (top of its page) to follow everything that happens in it.

CHAPTER 8

Publish a Website with GitHub Pages

GitHub Pages turns a repository into a free public website, which is perfect for portfolios, documentation, and small business sites.

PUBLISH A SITE

1. **Put your site in a repository.**
   At minimum, an `index.html` file at the top level.
2. **Open the repository's settings.**
   Click **Settings**, then **Pages** in the left sidebar.
3. **Choose the source.**
   Under **Build and deployment**, set **Source** to **Deploy from a branch**.
4. **Pick the branch.**
   Choose `main` and the `/ (root)` folder, then click **Save**.
5. **Wait a minute or two.**
   Refresh the Pages settings; your address appears at the top: `https://your-username.github.io/repository-name/`.

> **NOTE**
> On the Free plan, Pages works for **public** repositories. A repository named `your-username.github.io` becomes your main site at `https://your-username.github.io`. You can also connect your own domain name under **Custom domain**.

CHAPTER 9

Automate with GitHub Actions

GitHub Actions runs tasks automatically when something happens in your repository: test the code on every push, publish a site, or send a message when an issue opens. Each automation is a **workflow**, written in a **YAML** file (a simple text format of names and values, indented with spaces) inside the folder `.github/workflows/`.

.GITHUB/WORKFLOWS/CHECK.YML

```yaml
name: Check my site
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Say hello
        run: echo "Running checks on ${{ github.ref_name }}"
```

Push that file and click the **Actions** tab to watch it run. A green check mark means it passed; a red X means it failed (click it to see why).

Starting from scratch? The **Actions** tab offers ready-made workflow templates for most languages and hosting services.

PLAN	INCLUDED ACTIONS MINUTES PER MONTH (PRIVATE REPOS)
Free	2,000
Pro / Team	3,000
Enterprise	50,000

Actions on **public** repositories using GitHub's standard runners are free.

> **TIP**
>
> You rarely need to write workflows by hand. Ask your AI tool: "add a GitHub Actions workflow that runs my tests on every pull request."

CHAPTER 10

GitHub and AI

10.1 GitHub Copilot

GitHub Copilot is GitHub's own AI assistant. It suggests code as you type in VS Code and other editors, answers questions in a chat panel, reviews pull requests, and, as the **Copilot coding agent**, can take an issue you assign to it, work in the background, and open a pull request.

COPILOT PLAN	PRICE	HIGHLIGHTS
Free	$0	2,000 code completions/month, limited chat and agent use
Pro	$10/month	Unlimited completions; coding agent; monthly AI credits
Pro+	$39/month	More AI credits; third-party agents such as Claude and Codex
Max	$100/month	The largest monthly AI credit allowance
Business / Enterprise	Contact sales	Organization controls and policies

Prices from github.com/features/copilot/plans, October 2026. Copilot measures heavier use in **AI credits** (1 credit = $0.01).

10.2 Claude Code and other agents

AI coding agents use GitHub as their shared workspace:

- **Claude Code** runs `git` and `gh` commands for you. Ask "commit this and open a pull request." Its cloud sessions at claude.ai/code clone your repositories and push branches back.
- Run `/install-github-app` inside Claude Code to install the Claude GitHub App, then mention `@claude` in an issue or pull request to have it do the work.
- **Cursor, Lovable, Bolt, Replit,** and similar tools can connect your GitHub account to save projects to a repository.

10.3 Codespaces: a computer in the cloud

Codespaces gives you a ready-to-code environment in your browser, with a full VS Code editor, terminal, and tools, running on GitHub's servers. On a repository, click **Code**, choose the **Codespaces** tab, then **Create codespace on main**. Personal accounts get 120 core-hours and 15 GB of storage free each month (a 2-core machine uses 2 core-hours per hour).

> **TIP**
>
> Stop codespaces you aren't using. Click **Code**, then **Codespaces**, then the **…** menu, then **Stop codespace**. Idle ones stop automatically after 30 minutes by default, but they still use storage until deleted.

CHAPTER 11

Security and Good Habits

11.1 Never commit secrets

A **secret** is anything that grants access: passwords, API keys, tokens. Once pushed, assume a secret is compromised, even if you delete it afterwards, because it remains in the history.

- Keep secrets in a `.env` file and list `.env` in `.gitignore`.
- For Actions, store secrets in `Settings` › `Secrets and variables` › `Actions`.
- GitHub's **push protection** blocks many common secret formats from being pushed to public repositories, and you can turn it on for others under `Settings` › `Advanced Security` (called **Code security** on some accounts).

> **LEAKED A KEY?**
> Revoke or rotate it immediately at the service that issued it (for example, generate a new API key and delete the old one). Removing it from GitHub is not enough.

11.2 Personal access tokens

Some tools ask for a **personal access token** (PAT), a password-like string that lets a program act on your behalf. Prefer **fine-grained** tokens, which you can limit to specific repositories and permissions:

CREATE A FINE-GRAINED TOKEN

1. **Open developer settings.**
   Profile picture, then `Settings` › `Developer settings` › `Personal access tokens` › `Fine-grained tokens`.
2. **Click Generate new token.**
3. **Name it and set an expiration**, for example 90 days.

4. **Choose repository access**
   Only select repositories is safest.

5. **Pick permissions.**
   Grant only what the tool needs (for example, **Contents: Read and write** and **Pull requests: Read and write**).

6. **Generate and copy the token.**
   You'll only see it once. Store it in a password manager.

11.3 Keep dependencies healthy

Turn on **Dependabot** (`Settings` › `Advanced Security`) to get alerts, and automatic pull requests, when a library your project uses has a known security problem.

11.4 Checklist for a healthy repository

- [ ] A README explaining what the project is and how to run it
- [ ] A `.gitignore` that excludes secrets and build files
- [ ] `main` protected by a ruleset requiring pull requests
- [ ] Dependabot alerts on
- [ ] 2FA on your account, recovery codes stored safely
- [ ] Tokens scoped to the minimum and set to expire

CHAPTER 12

Plans and Pricing

PLAN	PRICE	INCLUDES
Free	$0	Unlimited public and private repos and collaborators; 2,000 Actions minutes and 500 MB Packages storage per month
Pro (individual)	$4/month	3,000 Actions minutes, 2 GB Packages, protected branches and Pages on private repos
Team	$4 per user/month	Pro features for organizations, plus team access controls
Enterprise	$21 per user/month	50,000 Actions minutes, 50 GB Packages, SSO, advanced compliance

From github.com/pricing and GitHub Docs, October 2026. Copilot and Codespaces are billed separately (Chapter 10).

> **TIP**
>
> The Free plan is plenty for learning, personal projects, and most small businesses.

CHAPTER 13

Troubleshooting

PROBLEM	FIX
`fatal: not a git repository`	You're not inside a repository folder. `cd` into it, or run `git init`.
`Authentication failed` / asked for a password	Sign in with `gh auth login`, GitHub Desktop, or Git Credential Manager. Your account password won't work.
`rejected … (fetch first)` when pushing	Someone else pushed first. Run `git pull`, fix any conflicts, then `git push`.
`Please tell me who you are`	Run the two `git config --global user.…` commands from Chapter 4.
Committed to the wrong branch	Ask your AI agent, or run `git switch -c new-branch` to keep the work on a new branch before switching back.
Accidentally committed a secret	Revoke the secret at its source first, then remove it from the repo.
Pages site shows 404	Check that `index.html` is in the folder you chose, wait a few minutes, and check the Actions tab for errors.

Quick Reference

Set up once: `git config --global user.name "Name"` · `git config --global user.email "you@example.com"` · `gh auth login`

Start: `git clone <url>` · or `git init` then `gh repo create <name> --private --source=. --push`

Daily loop: `git status` → `git add .` → `git commit -m "message"` → `git push` · Start of day: `git pull`

Branch: `git switch -c <name>` · `git push -u origin <name>` · `gh pr create` · back to main: `git switch main`

Look back: `git log --oneline` · `git diff` · undo a commit safely: `git revert <id>`

Website: Settings › Pages → Deploy from a branch → `main` / root

Glossary

.gitignore	A file listing which files Git should never track.
Branch	A separate line of development, letting you change things without affecting the main version.
Clone	Download a complete copy of a repository, including its history, to your computer.
Commit	A saved snapshot of your files with a message explaining the change.
Fork	Your own copy of someone else's repository on GitHub, used to propose changes.
Git	Free software that tracks the history of files in a folder.
GitHub	A website that hosts Git repositories and adds collaboration tools.
GitHub Actions	GitHub's automation system for running tasks when events happen in a repository.
GitHub CLI	The `gh` command-line program for working with GitHub from a terminal.
GitHub Pages	Free website hosting straight from a repository.
Issue	A tracked to-do, bug report, or idea on GitHub.
Main	The default branch, which holds the official version of a project.
Markdown	A simple text formatting language used across GitHub.
Merge	Combine the changes from one branch into another.
Merge conflict	When two branches changed the same lines differently, and a person must choose the result.
Passkey	A passwordless sign-in method using your device's fingerprint, face, or PIN.

Personal access token	A password-like string that lets a program act on your GitHub account with limited permissions.
Pull	Download new commits from GitHub to your computer.
Pull request	A proposal to merge one branch into another, with review and discussion.
Push	Upload your commits from your computer to GitHub.
README	The front-page document of a repository, usually `README.md`.
Remote	A copy of the repository hosted elsewhere, usually GitHub, named `origin` by default.
Repository	A project folder tracked by Git, together with its full history.
Ruleset	Settings that protect branches, for example requiring pull requests before changes reach main.
Secret	Any password, key, or token that grants access and must be kept private.
Staging area	The list of changes chosen to go into the next commit.
Two-factor authentication	Signing in with a password plus a second proof, such as a phone code.
Workflow	An automation defined in a YAML file for GitHub Actions.
YAML	A plain-text format for settings files, using indentation and `name: value` pairs.